Experiment 2: what kind of problem is it?

A plain-language walk through why we ran E2, how we ran it so the results can be trusted, what came out, and what it means for the paper. Every number is computed from the committed result files by `scripts/e2_make_report.py`.

THE SHORT VERSION

GO to E3, narrowed to *when can a depth hint be trusted*

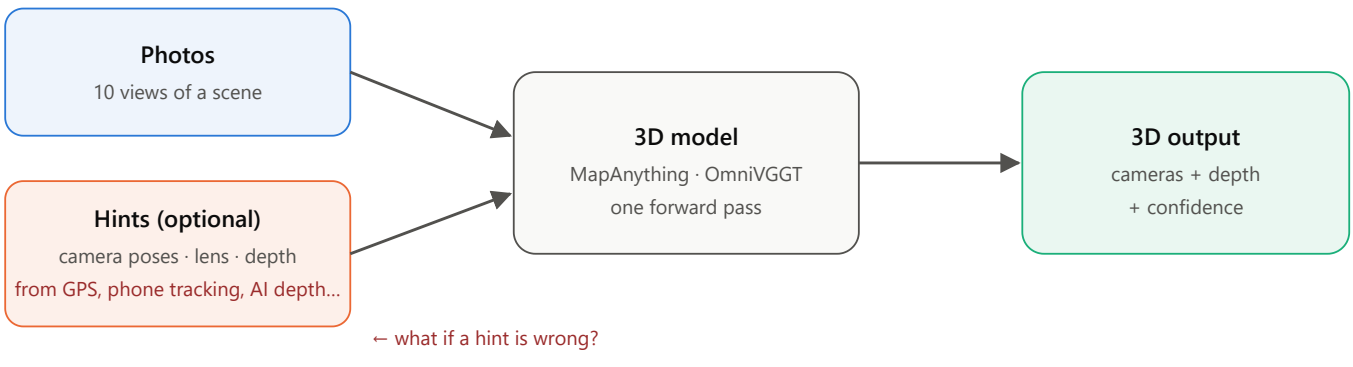
MapAnything copies wrong camera hints almost completely, even when its own confidence shows it knows they are wrong, and it does not notice errors that agree with each other. A second model, OmniVGGT, follows bad hints less (about 60%) and, by our strict test, is not made worse than photos-only by synthetic errors. In real data, phone sensors **help** both models, but **AI-estimated depth hurts both**. Existing fixes handle single, grossly wrong cameras; they do not handle bad depth or errors that agree. The decision follows the frozen rule, with two judgment calls spelled out in section 10. The final call belongs to the team.

13,674	0.93	0.93	+10.0 pp / +9.1 pp
model runs, 0 failures	MapAnything copies a 15° camera error (absorption)	its confidence still flags the bad camera (AUROC)	AI depth makes shape worse (MapAnything / OmniVGGT)

1. Background: what these models do, and what a "hint" is

Models like **MapAnything** and **OmniVGGT** take a handful of photos of a scene and, in one pass, output a 3D reconstruction: where each camera was, and how far away every pixel is. Their selling point is that they can also take **hints**: where the cameras were (poses), the lens settings (intrinsics) and depth maps. With correct hints, they get much better.

The catch: in the real world hints come from GPS, phone tracking, cheap depth sensors, AI depth estimators or wrong photo metadata, so they are often *wrong*. Our project asks what happens then.



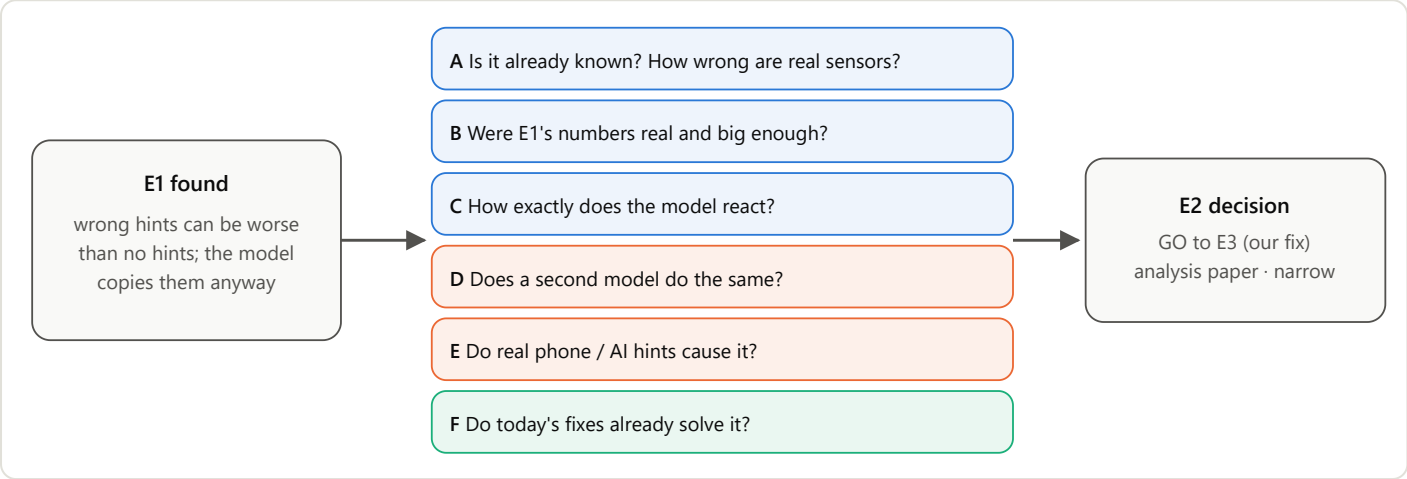
The setting: photos plus optional hints go into one model. Our question is what a wrong hint does.

> **How we score.** *Rotation error* = how wrong the camera directions are (degrees, between pairs of cameras). *Shape error* = how far the 3D points are from the truth after one overall size correction (% of distance). *Real-size error* = the same without the size correction. *Absorption* = how much of a hint's error ends up in the output (1 = copied, 0 = ignored). *AUROC* = how well the model's own confidence ranks bad cameras below good ones (0.5 = guessing, 1 = perfect).

2. Where E1 left us, and why E2 was needed

Experiment 1 (preregistered, run once on 10 test scenes) answered the first question: **yes, a wrong hint can make MapAnything worse than giving it no hints at all**. With one camera turned 15° wrong, its rotation error was +114% compared with photos only. The model copied the bad pose almost completely (absorption 0.95 at 15°), even though its own confidence did flag the bad camera.

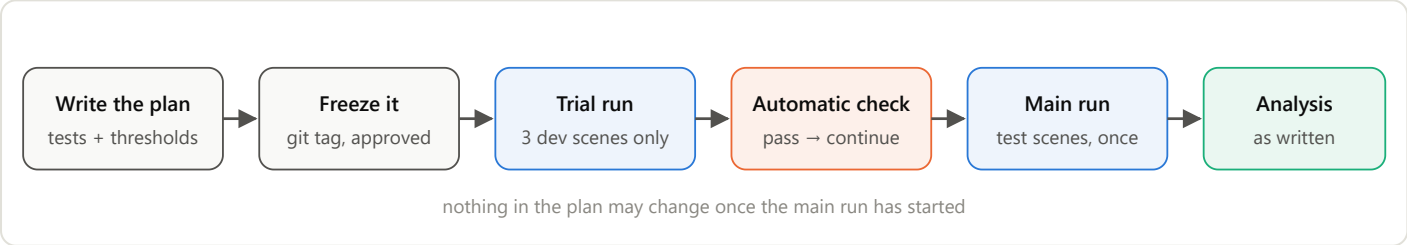
But E1 left the questions that decide whether a *fix* is worth building open. Is this already known? Were the numbers big enough to matter, or just statistically detectable? *Why* does the model obey a hint it seems to know is wrong: a simple averaging step, or something deeper? Is it only MapAnything? Do real sensors produce such errors? And do existing tools already fix it? A method paper only makes sense if the answers line up, so E2 asks them one by one.



E2's six parts, each answering one question that decides the next step.

3. How we ran E2, and why it was done this way

The biggest risk in an experiment like this is fooling ourselves: running many tests, then reporting the ones that happen to look good. So E2 followed strict rules, each with a reason.



Every main run followed this order. The plan is fixed and tagged in git before the main run.

- **Preregistration.** Before each main run we wrote down exactly what we would measure and what counts as a result, and froze it (git tags `e2-prereg-C`, `e2-prereg-DEF`). *So that* nobody can choose the tests after seeing the numbers.
- **Trial (dev) scenes vs test (eval) scenes.** Code is debugged only on 3 separate trial scenes; the test scenes are used once. *So that* the test data never shapes the design.

- **"Statistically clear" AND "big enough".** A result counts only if it survives Holm correction (which accounts for running many tests at once) *and* passes a practical threshold: shape ≥ 1 percentage point, real size ≥ 5 pp, rotation ≥ 0.25°, scale ≥ 5%. *So that* tiny, irrelevant effects never count as findings.
- **The scene is the unit.** Several clips from the same scene are averaged first. *So that* we don't pretend to have more independent evidence than we do.
- **Every comparison is paired.** The same clips, the same random corruptions, with and without the bad hint. *So that* differences come from the hint, not from different data.
- **One paid GPU session, no idle time.** All E2 GPU work ran in one session on an NVIDIA RTX PRO 6000 (2026-11-01 15:27:19 → 2026-11-01 17:34:01 UTC), with automatic checks between each trial run and its main run, so the GPU never waited for a human.
- **A parity check against E1.** The new GPU reproduced E1's runs: median differences 0.006 pp shape and 0.0017° rotation, and parallel processes gave bit-identical results. One extreme run (a 10× GPS jump) differed by 2.1 pp; you approved judging the check by its 99th percentile instead (Amendment 14).

---

4. E2-A: is this already known? (literature audit)

We opened every closely related paper. **No paper measures our core findings on these models.** The closest are Pow3R (two views only; it *refuses* grossly wrong hints), VPGGT (retrains OmniVGGT against drift-style pose noise; poses only; it doesn't detect which hint is bad), WorldMirror and TCO. We also collected realistic error sizes from the literature (phone tracking, compass, GPS, depth sensors, AI depth estimators), which replace E1's rough "realistic" labels. Details: `reports/e2_literature.md`.

---

5. E2-B: were E1's results real and big enough?

We checked that "copying" is genuine behaviour (the output camera comes from the network, not from passing the input through), then re-tested every E1 setting with the strict standard. **16 of 132 tests** stay harmful, covering **9 of 19 error types**; 7 of them at sizes that really occur (first camera wrong, one depth map scaled wrong, depth hints scaled wrong, every camera slightly turned, one camera turned wrong, all distances scaled wrong, same heading error on all). The harm sits on the bad camera itself: the pairs of good cameras barely change. Details: `reports/e2_audit.md`.

---

6. E2-C: how does MapAnything react to a bad hint?

We fed MapAnything carefully controlled bad hints on 24 clips from 10 test scenes (2026-11-01, finished 17:06 UTC; 3,232 runs) and asked twelve preregistered questions.

It copies, and it never overrides

Turn one camera's hint by an increasing amount and measure how much of that error appears in the output. A sensible model would follow small corrections and refuse absurd ones. MapAnything does the opposite of refusing: it copies 0.77 of a 1° error and 0.96 of a 30° error. The same holds for position errors. OmniVGGT copies about half of a rotation error at any size, and almost none of a position error.

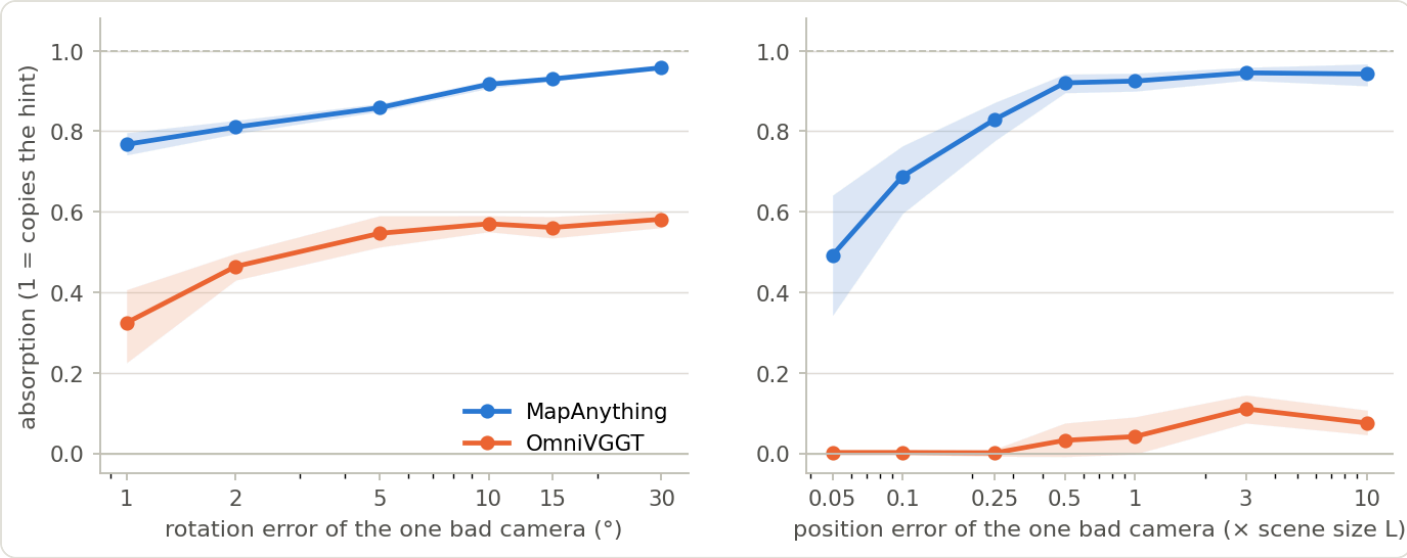


Figure 1. How much of a bad hint each model copies, as the error grows (shaded: 95% CI over scenes). Neither curve turns down: no model overrides an absurd hint.

Source: `results/e2c_eval/agg/c1_c3_trust_curves.csv; OmniVGGT from results/e2d_eval`

It knows, but obeys

Yet MapAnything's own confidence clearly marks the bad camera (AUROC 0.93). The information needed to resist is inside the model; it just doesn't use it. OmniVGGT's confidence barely notices (AUROC 0.58).

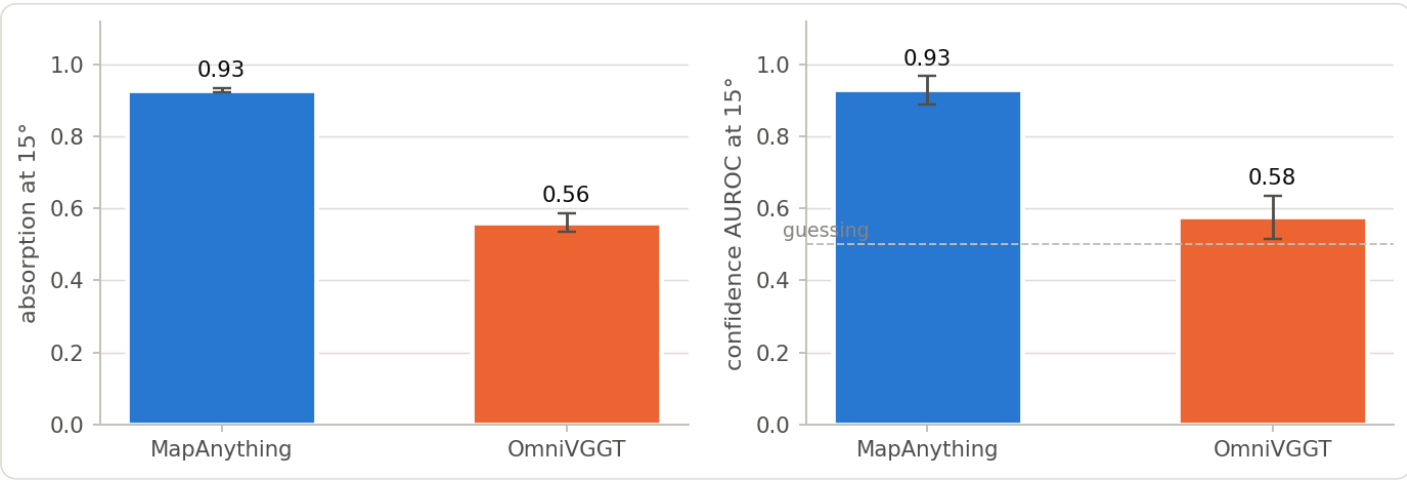


Figure 2. Left: share of a 15° error copied. Right: how well the model's own confidence spots the bad camera (dashed: guessing).

Source: `results/e2c_eval/agg/primary_tests.csv, results/e2def_agg/d_behaviour_omnivggt.csv`

Errors that agree are the dangerous ones

Give every camera a 5° error in random directions, and MapAnything's confidence drops to 0.28 of normal. Give every camera the *same* 5° error, and it only drops to 0.53. Drift behaves the same way. Errors that agree look self-consistent, so the model doesn't notice them. Real phone tracking errors are exactly this kind (section 8). OmniVGGT's confidence doesn't move at all with bad camera hints (ratio 1.00), so it gives no warning in either case.

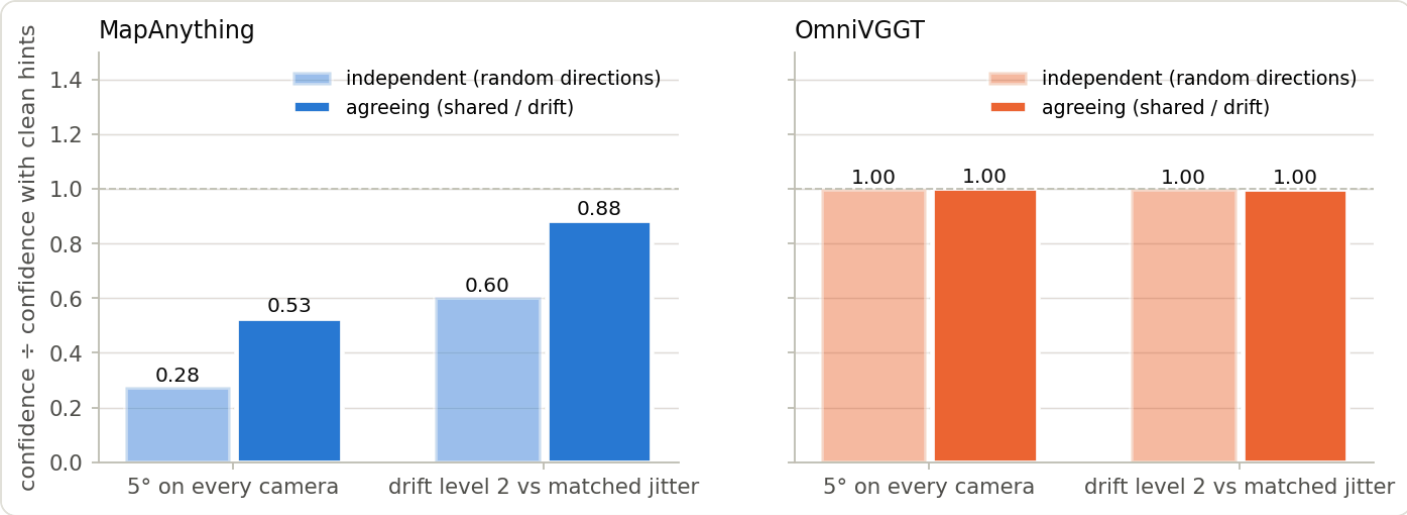


Figure 3. Average confidence relative to clean hints. Pale: independent errors; solid: agreeing errors of the same size. Source: `results/e2c_eval/agg/c6_correlated_vs_independent.csv; OmniVGGT via the same function`

Why it happens: not the averaging step

MapAnything rescales camera hints using their *mean* distance. A single crazy hint can wreck a mean but not a median, so one theory (our H2) was that the averaging step causes the damage. We swapped it for a median. Clean results didn't change (+0.0054° rotation), and the 10× GPS jump wasn't fixed (P7: -0.84 pp, not clear). **The damage happens inside the network.**

All twelve questions

Test	Question	Measured as	Finding if	Result [95% CI]	Holm p	Verdict
P1	Does it stop following a grossly wrong rotation (30° vs 15°)?	absorption difference	≤ −0.10	+0.028 [+0.023, +0.032]	0.0000	✗ not a finding
P2	...or a grossly wrong position (10 L vs 0.25 L)?	absorption difference	≤ −0.10	+0.112 [+0.080, +0.144]	0.0012	✗ not a finding
P3	Does its own confidence flag the bad camera?	AUROC − 0.5	≥ 0.15	+0.431 [+0.388, +0.469]	0.0000	✓ finding
P4	Does a wrong rotation bend that camera's own 3D shape?	Δ shape error	≥ 1 pp	+0.006 [+0.002, +0.009]	0.1207	✗ not a finding
P5	...or bend the good cameras' shape?	Δ shape error	≥ 1 pp	+0.000 [-0.000, +0.001]	0.4847	✗ not a finding
P6	Does a median scale step change clean results?	Δ rotation error	|Δ| ≥ 0.25°	+0.005 [-0.002, +0.016]	0.6184	✗ not a finding
P7	Does a median scale step fix a 10 L GPS jump?	Δ shape error	≤ −1 pp	-0.008 [-0.017, -0.002]	0.4847	✗ not a finding
P8	Does it copy less where the photos show the scene well?	Spearman ρ	≤ −0.30	-0.186 [-0.367, -0.017]	0.4847	✗ not a finding
P9	Does it copy less with more photos (20 vs 4)?	absorption difference	≤ −0.10	+0.005 [-0.006, +0.014]	0.6184	✗ not a finding
P10	Does it notice a shared error less than random errors?	confidence ratio difference	≥ +0.10	+0.252 [+0.173, +0.332]	0.0022	✓ finding
P11	Does it notice drift less than matched jitter?	confidence ratio difference	≥ +0.10	+0.280 [+0.163, +0.423]	0.0272	✓ finding
P12	Does one bad hint weigh more when few poses are given?	Δ rotation harm	≥ 0.25°	+0.132 [-0.006, +0.267]	0.4847	✗ not a finding

Source: `results/e2c_eval/agg/primary_tests.csv`. Holm correction over all 12 tests. A non-finding is not proof of absence; for P1 and P2 the confidence intervals sit well above the override threshold, so any override, if it exists, is smaller than the one we looked for.

---

7. E2-D: does a second model behave the same?

A finding that holds for two differently built models is a property of the approach, not one model's quirk. OmniVGGT injects hints differently, so we gave it *identical* bad inputs: the same clips, and the same condition names, which means the same random corruptions (3,312 runs).

Result: under the strict test, none of the 9 harmful error types makes OmniVGGT worse than photos only. Two reasons. First, it copies less (about 60% instead of about 95%). Second, its photos-only mode is unstable: it fails badly in 2 of 10 scenes (terrains 32.6°, meadow 16.8°), so "worse than photos only" becomes too noisy to prove. With one camera turned 90°, OmniVGGT *is* worse than photos only in 8 of 10 scenes, but the strict test doesn't pass. It is also immune by design to wrong overall scale, because it rescales every hint.

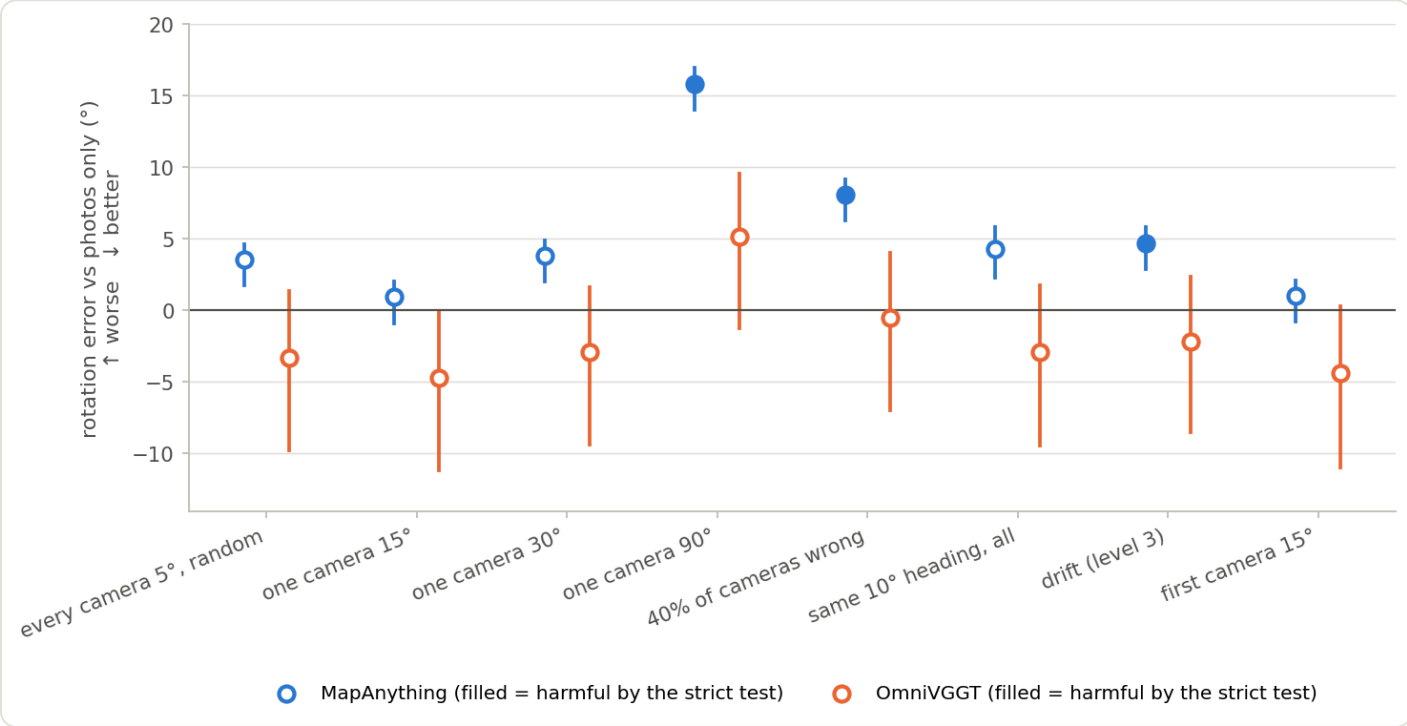


Figure 4. Rotation error with each bad hint, minus photos-only (95% CI over scenes). Filled markers pass the strict test. Source: `results/e2def_agg/d_harm_*.csv`

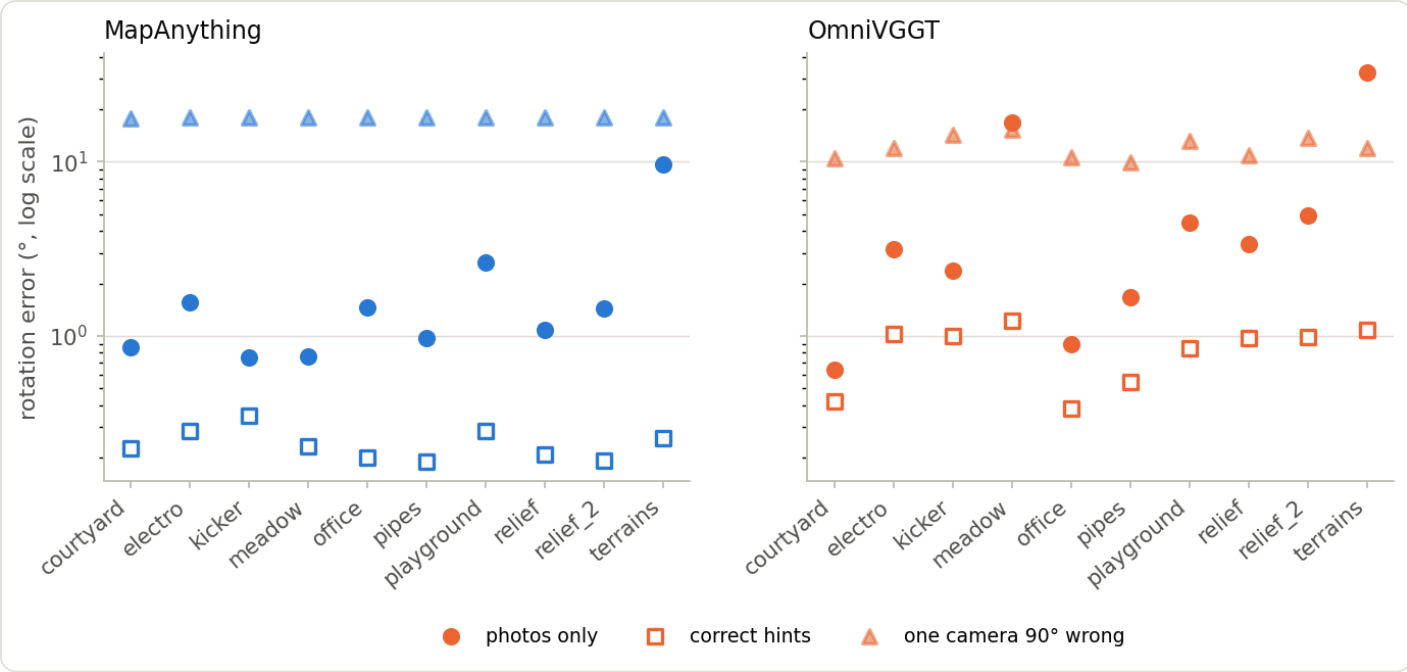


Figure 5. Per scene: photos only, correct hints, and one camera 90° wrong. OmniVGGT's photos-only result is unstable, which makes 'worse than photos only' hard to prove for it.

Source: `results/e2df_eval, results/e2d_eval per_run.csv`

Error type	Harmful for MapAnything (setting)	Harmful for OmniVGGT	Class
first camera wrong	none	none	• neither
one depth map scaled wrong	1.5 (shape)	none	• MapAnything only
depth hints scaled wrong	1.5 (real_size), 1.5 (scale_ratio)	none	• MapAnything only
odometry drift	3 (rotation)	none	• MapAnything only
every camera slightly turned	none	none	• neither
several cameras wrong	0.4 (rotation)	none	• MapAnything only
one camera turned wrong	90 (rotation)	none	• MapAnything only
all distances scaled wrong	2 (scale_ratio)	none	• MapAnything only
same heading error on all	none	none	• neither

Behaviour test	MapAnything		OmniVGGT		Class
P1	+0.028	✗ no	+0.020	✗ no	neither
P2	+0.112	✗ no	+0.074	✗ no	neither
P3	+0.431	✓ finding	+0.079	✗ no	MapAnything only
P4	+0.006	✗ no	+0.000	✗ no	neither
P5	+0.000	✗ no	+0.000	✗ no	neither
P10	+0.252	✓ finding	+0.002	✗ no	MapAnything only
P11	+0.280	✓ finding	-0.002	✗ no	MapAnything only

Sources: `results/e2def_agg/d_family_class.csv`, `d_side_by_side_behaviour.csv`. E1-definition absorption at 5/15/30/90°: MapAnything 0.88, 0.95, 0.97, 0.99; OmniVGGT 0.57, 0.58, 0.57, 0.60.

8. E2-E: do real sensors cause the problem?

Everything so far used errors we injected on purpose. Reviewers will rightly ask whether real sensors produce them. So we used **SceneFun3D**: iPad recordings of real rooms with the iPad's own tracking (ARKit) poses and LiDAR depth, plus much better laser-scan ground truth. They come from the ARKitScenes validation split, which neither model trained on. We also added a hint that many cameras without LiDAR would really use: **AI-estimated depth** (Depth Anything V2, metric indoor model).

> **Data care.** 18 rooms (3 trial, 15 test), 3 clips each, chosen by a fixed random rule. Only the frames we use were downloaded (~60 MB per video instead of ~650 MB). Some phones store frames turned 180°; this is detected per video and corrected, and the correction reproduces the dataset's own lens values to within 0.1 pixel. 5 portrait recordings were skipped, and 5 clips failing the ground-truth check were excluded, both by rules fixed before any model ran. One test clip turned out to have a broken ground-truth pose for one view (a 164° jump in half a second; the phone's own pose is fine). It stays in the main analysis as preregistered; leaving it out changes 0 verdicts.

How wrong are real hints?

Phone poses are good: 0.87° typical rotation error, and only 1.4% of views 5° or more off. Their errors build up along the recording (median correlation with view order 0.52), so they are drift-like and agreeing. LiDAR depth is excellent (×0.996, 1.2% typical error). AI depth is not: typically ×1.42 too far (range ×1.13–×1.85), with 42% typical error.

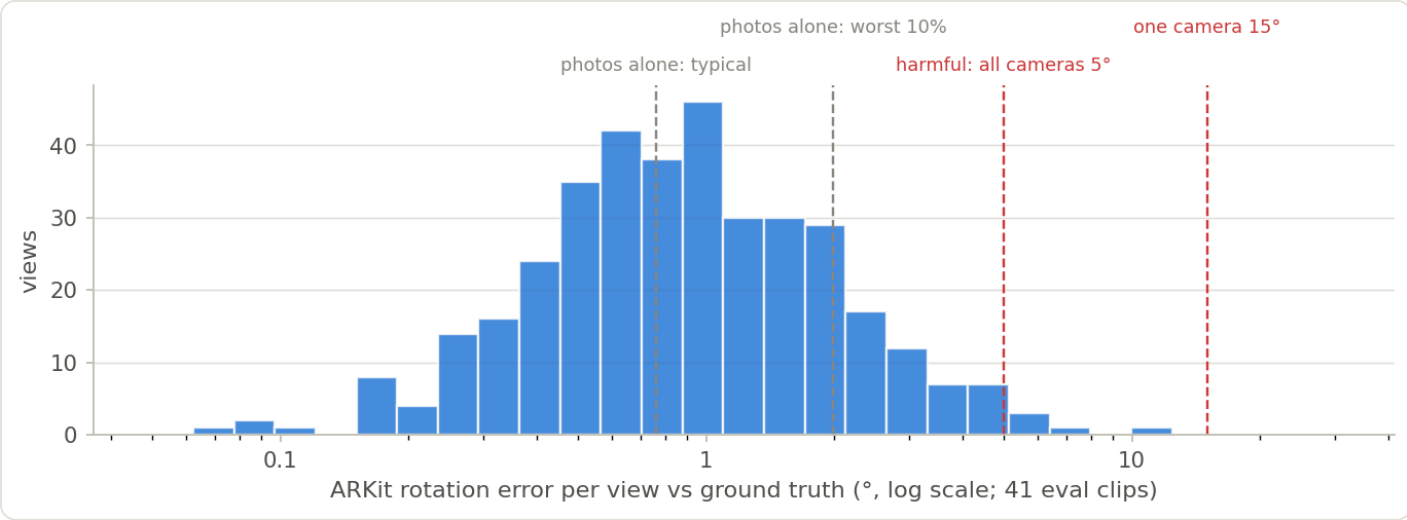


Figure 6. Real phone rotation errors per view, against what photos alone can see (grey) and the sizes E2-B found harmful (red).

Source: `data/clips_e2e (dev_poses vs poses), results/e2c_eval/agg/c8_photos_alone.csv`

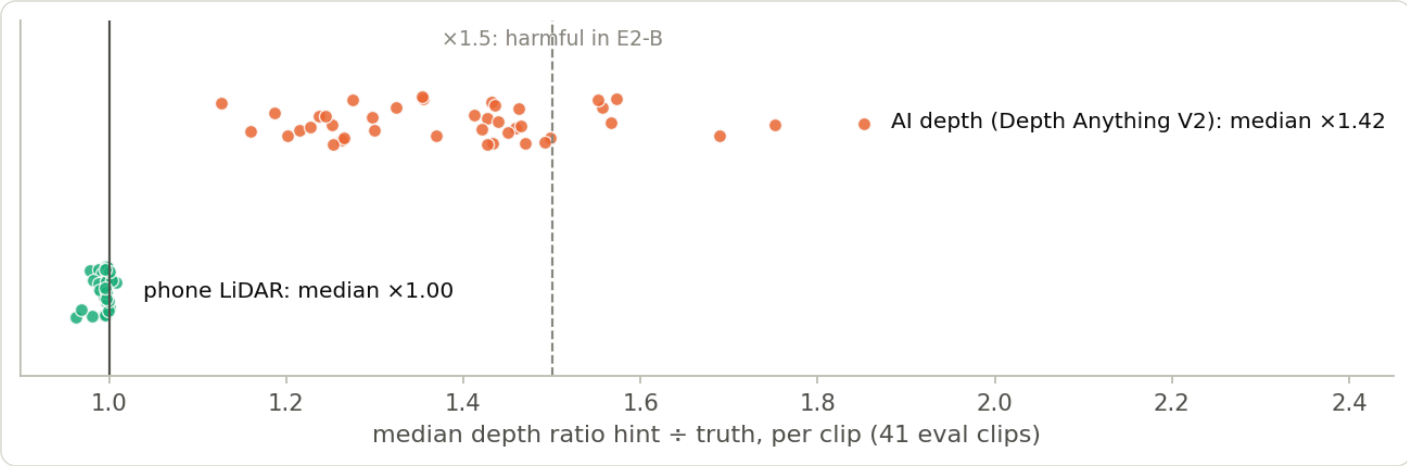


Figure 7. Depth scale error per test clip. LiDAR sits on the truth; AI depth is 1.13–1.85× too far.
Source: `results/e2e_mono/depth_hint_errors.csv`

Do real hints help or hurt?

Real phone hints help. AI depth hurts both models, by about 10 percentage points of shape error and about 1° of rotation, and good phone poses don't cancel it out.

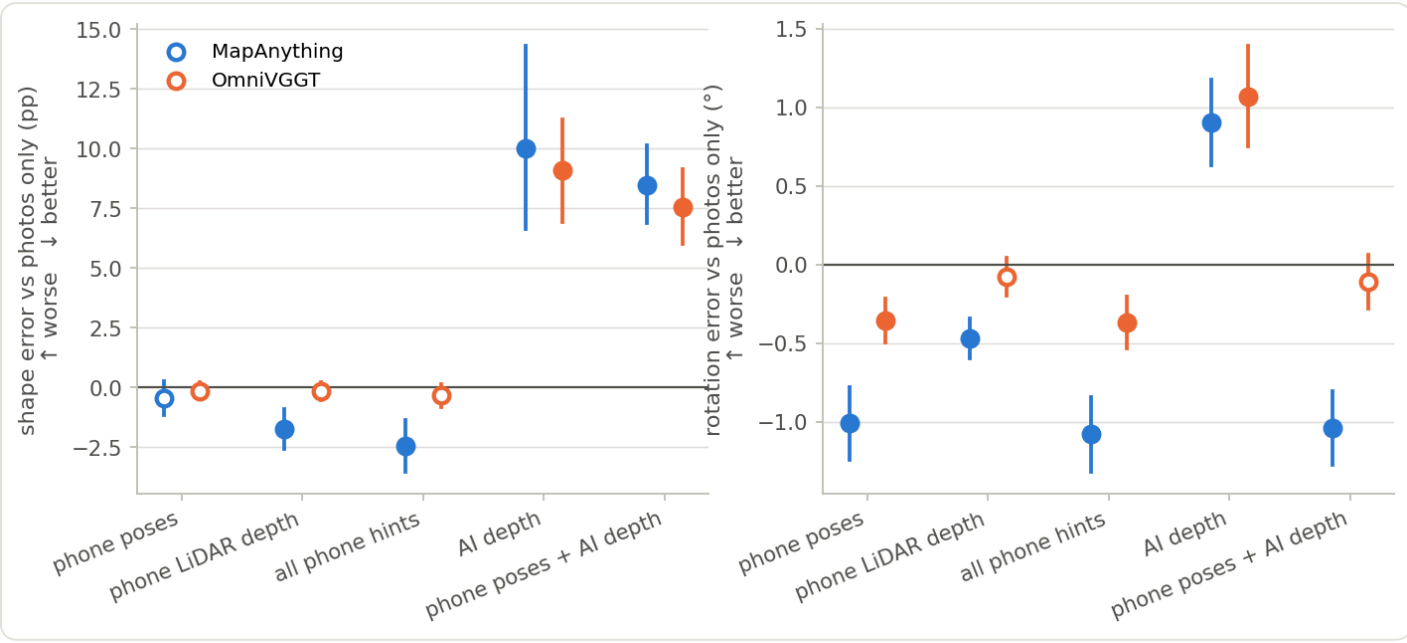


Figure 8. Each real hint compared with photos only (95% CI over 15 rooms). Filled markers pass the strict test.
Source: `results/e2def_agg/e_tests.csv`

Hint	Model	Shape	Rotation	Real size
phone lens values	MapAnything	-0.9 pp • small	-0.18° • small	–
phone poses	MapAnything	-0.4 pp • unclear	-1.01° ▼ helps	-31.4 pp ▼ helps
phone poses	OmniVGGT	-0.1 pp • unclear	-0.36° ▼ helps	–
phone LiDAR depth	MapAnything	-1.7 pp ▼ helps	-0.47° ▼ helps	-36.8 pp ▼ helps
phone LiDAR depth	OmniVGGT	-0.1 pp • unclear	-0.08° • unclear	–
all phone hints	MapAnything	-2.4 pp ▼ helps	-1.08° ▼ helps	-37.5 pp ▼ helps
all phone hints	OmniVGGT	-0.3 pp • unclear	-0.37° ▼ helps	–
AI depth	MapAnything	+10.0 pp ▲ hurts	+0.90° ▲ hurts	+3.4 pp • unclear
AI depth	OmniVGGT	+9.1 pp ▲ hurts	+1.07° ▲ hurts	–

Hint	Model	Shape	Rotation	Real size
phone poses + AI depth	MapAnything	+8.5 pp ▲ hurts	-1.04° ▼ helps	-8.5 pp • unclear
phone poses + AI depth	OmniVGGT	+7.6 pp ▲ hurts	-0.11° • unclear	–

Source: `results/e2def_agg/e_tests.csv` (Holm over 27 tests). "small" = statistically clear but below the practical threshold. Sensitivity without the broken-ground-truth clip: `results/e2def_agg/sensitivity/`.

---

9. E2-F: do today's fixes already solve it?

A method paper must show the problem is not already solved. For each setting that is clearly harmful for MapAnything, we measured how much of the damage is *left* after each existing fix: 0 = fully fixed, 1 = no help, above 1 = the fix is worse than the damage. R0 ignores all hints; R1 drops the pose hints; R2 is E1's simple filter (drop poses that disagree with the photos-only answer); R3 drops cameras the model itself is unsure of; ORACLE cheats by dropping exactly the bad cameras.

Bad hint (setting)	Metric	R0 photos only	R1 drop poses	R2 E1 filter	R3 confidence	ORACLE	drop depth*	Best of R0–R3	Damage left
outlier_rot_rot_deg=90	rotation	0.11	0.04	0.01	0.34	0.01	–	R2 E1 filter	0.01
outlier_frac_frac=0.4	rotation	0.19	0.07	0.05	0.98	0.04	–	R2 E1 filter	0.05
scale_all_factor=2	scale_ratio	0.25	0.21	0.93	1.00	0.22	–	R1 drop poses	0.21
drift_level=3	rotation	0.29	0.10	0.42	0.95	0.12	–	R1 drop poses	0.10
depth_scale_factor=1.5	real_size	2.13	3.27	–	–	–	0.10	R0 images only	2.13
depth_scale_factor=1.5	scale_ratio	1.35	2.48	–	–	–	0.12	R0 images only	1.35
depth_one_scale_factor=1.5	shape	4.17	5.90	–	–	–	0.15	R0 images only	4.17

Source: `results/e2def_agg/f_remaining_gap.csv`. *"Drop depth hints" is exploratory, not preregistered.

What this says. For a single, grossly wrong camera, E1's filter is nearly perfect, but it also drops 12% of *good* cameras (R3: 3%). On real phone data that costs accuracy: the filter makes rotation *worse* by +0.19° because it throws away good real hints. For agreeing errors (drift, scale), the model's confidence is useless (R3 leaves 90–100%); only "drop all poses" helps, which also throws away the value of good poses. For bad depth, none of the preregistered fixes helps; simply dropping the depth hint would, *if you knew it was bad*. But phone LiDAR depth helps, so dropping depth always is not free.

What photos alone can reveal (the ceiling)

Any fix that uses only the photos can't detect an error smaller than the photos' own uncertainty. With photos and lens values only, camera directions are about 0.8° uncertain (2.0° in the worst 10%), and overall scale about 13% (52% in the worst 10%). So typical phone errors (about 1°) are invisible to photos, errors of 5° or more are visible, and AI depth's ~42% scale error is visible in most clips.

10. The decision

The E2 plan fixed the decision rule in advance. To GO to E3, at least one harmful error type must pass all four tests: (a) it hurts both models, (b) it happens with real sensors, (c) the best existing fix leaves at least half the damage, and (d) photos alone contain enough evidence to detect it.

Error type	(a) both models	(b) real sensors	(c) ≥ half left	(d) visible	Passes
Wrong camera rotations (one bad camera, several, drift, shared heading)	✗ MapAnything only	✗ phone errors too small	✗ fixes leave 1–10%	✓ large ones	✗ no
Wrong overall scale	✗ MapAnything only	• not tested	✗ fixes leave 21%	✓ yes	✗ no
Wrong depth hints (AI depth)	✓ both models (real data)	✓ AI depth	✓ by the frozen rule*	✓ mostly	✓ YES

Verdict: GO to E3, narrowed to depth hints. Two judgment calls behind it, stated openly:

1. **Test (a) for depth comes from the real AI-depth runs.** The synthetic depth-scale test is "MapAnything only", because OmniVGGT rescales depth. But real AI depth also distorts the *shape* of the depth map, and that hurts both models.
2. **Test (c) holds by the frozen rule, but an obvious fix was missing from the list.** "Drop the depth hint" (exploratory) leaves only 10–15% of the damage. It works only if you *know* the depth is bad; LiDAR depth helps, so dropping it always loses accuracy. The real open problem is deciding *when* to trust a depth hint, which is exactly E3's question.

Read strictly the other way, the outcome would be the **analysis paper**, which is already well supported.

11. What this means for the paper and for E3

- **Already solid (analysis paper):** these models copy wrong hints instead of overriding them; MapAnything does so while its confidence knows; agreeing errors go unnoticed; the cause is inside the network, not the averaging step; real AI depth hurts both models while real phone sensors help; existing fixes only cover the easy case.
- **E3 target:** decide from the photos alone, with no ground truth, whether a hint (first of all a depth hint) can be trusted, and use it only then. E3 must beat three simple policies: always use hints, never use them, and E1's filter. Agreeing pose errors are a secondary target, since nothing deployable handles them today.
- **Risks:** OmniVGGT is less affected by synthetic errors; typical phone errors (~1°) are below what photos can detect; and time (CVPR submission on 16 November 2026).

Appendix: reproducibility and where things are

- Plans and amendments: `experiments/E2.md` (Amendments 1–14), data plan `experiments/E2E_data.md`. Frozen code: tags `e2-prereg-C` (E2-C) and `e2-prereg-DEF` (E2-D/E/F).
- Machine: NVIDIA RTX PRO 6000 Blackwell Server Edition, 580.126.09, 97887 MiB. MapAnything weights snapshot `a1d87e90`, OmniVGGT `29c67625`, Depth Anything V2 revision recorded in `results/e2e_mono/mono_depth.csv`.
- Runs: 13,674 in total, 0 failures; queue log `results/vm_logs_e2/session.txt`.

- Analyses: `python -m exp1.e2c_analysis …` → `results/e2c_eval/agg/`; `python -m exp1.e2def_analysis …` → `results/e2def_agg/`. This report: `scripts/e2_make_report.py`.
- Not in E2: R4 (robust bundle adjustment, CPU only) is still to run; wrong lens values could not be tested on real data (the dataset's lens values come from the phone); portrait recordings are excluded.